LATIHAN TERSTRUKTUR (TUGAS KULIAH)

Pertemuan 4:
Implementasi Unit Testing dengan GitHub Actions

Nama	:	Diva Anastasya
NIM	:	1224026
Kelas	:	Implementasi Pengujian P/L
Tanggal	:	12 Oktober 2026
Repository	:	github.com/diva-28-ukay/latihan-git-1224026
File	:	Tugas04_GitHubActions_1224026_DivaAnastasya.pdf

Struktur laporan: Cover - Daftar Isi - Bagian A (Git & GitHub, 25p) - Bagian B (Unit Testing, 35p) - Bagian C (GitHub Actions, 25p) - Bagian D (Coverage, 15p) - Bagian E Bonus BankAccount (10p) - Lampiran & Link Repo. Kode lengkap ada di repo, laporan ini memuat jawaban teori + potongan kode + perintah verifikasi.

Penilaian: A 25 + B 35 + C 25 + D 15 + E bonus 10 = 100 (+10). Deadline minggu ke-5 sebelum kuliah. Kumpul: folder Tugas/Tugas04 + submit link ke LMS.

DAFTAR ISI

- Bagian A: Git & GitHub Dasar - Soal 1 (teori) & Soal 2 (praktik 10 langkah)
- Bagian B: Unit Testing - Soal 3 (PendaftaranValidator, 5 TC + JUnit) & Soal 4 (StudentGrade, CC=7, 7 path + JUnit)
- Bagian C: GitHub Actions - Soal 5 (analisis Python CI) & Soal 6 (workflow Java Unit Test)
- Bagian D: Code Coverage - Soal 7 (50 stmts, 11 miss, payment 60%)
- Bagian E Bonus: BankAccount + 9 test + workflow + pipeline
- Lampiran: struktur repo, cara run mvn test, link GitHub

LOKASI FILE DI REPO

File	Isi / Soal
README.md	Nama + NIM
test.txt	Soal 2 (feature-test)
src/main/.../PendaftaranValidator.java	Soal 3
src/main/.../StudentGrade.java	Soal 4
src/main/.../BankAccount.java	Soal 8 bonus
src/test/.../*Test.java (3 file)	JUnit Soal 3, 4, 8
.github/workflows/java-unit-test.yml	Soal 6 + 8
JAWABAN_TUGAS04.md	Jawaban markdown
Tugas04_..._DivaAnastasya.pdf	Laporan ini

BAGIAN A: GIT & GITHUB DASAR (25 POIN)

Soal 1 (10 Poin) - Teori

1. Centralized VCS vs Distributed VCS

Centralized VCS (CVCS): hanya ada SATU server pusat yang menyimpan seluruh riwayat. Client checkout working copy, setiap commit langsung ke server. Butuh koneksi untuk commit/log, single point of failure.

Distributed VCS (DVCS): setiap clone menyimpan SALINAN PENUH history. Bisa commit offline, sinkron via push/pull/merge. Branching murah, tahan kegagalan server.

Aspek	Centralized	Distributed
Penyimpanan history	Hanya di server	Di setiap clone
Commit offline	Tidak bisa	Bisa
Contoh	SVN, CVS (+ Perforce)	Git, Mercurial (+ Bazaar)
Kelebihan	Akses terpusat sederhana	Offline, cepat, branching murah
Kelemahan	Tergantung server	Konsep awal sedikit kompleks

2. Lima Git command wajib

Command	Fungsi
git clone <url>	Salin repo remote ke lokal + full history
git add <file> / .	Masuk staging area, siap commit
git commit -m "pesan"	Simpan snapshot staging ke history lokal
git push origin <branch>	Kirim commit lokal ke GitHub
git pull	Ambil + gabung remote (fetch + merge)

Bonus yang juga wajib: git status (cek state), git branch (list/buat), git checkout -b / switch (pindah), git merge (gabung), git log --oneline --graph (riwayat).

3. Fungsi .gitignore

File .gitignore berisi pola file/folder yang DIABAIKAN Git: tidak di-track, tidak ikut add/commit/push. Fungsi: cegah file sampah/privat masuk repo.

```
target/
*.class
*.jar
.idea/
.vscode/
*.iml
.env
*.exec
```

Di project ini .gitignore mengabaikan target/, *.class, IDE, OS, dan hasil coverage.

Soal 2 (15 Poin) - Praktik 10 langkah (NIM 1224026)

Langkah 1 dibuat via web GitHub (New repository > latihan-git-1224026). Sisanya via CLI. Semua sudah dipraktikkan di repo ini:

#	Tugas	Command
1	Buat repo latihan-git-[NIM]	Web: New repository (atau gh repo create)

#	Tugas	Command
2	Clone ke lokal	git clone https://github.com/diva-28-ukay/latihan-git-1224026.git
3	Buat README + nama/NIM	echo Nama/NIM > README.md
4	Commit "docs: add README"	git add README.md; git commit -m "docs: add README"
5	Push main	git push origin main
6	Buat branch feature-test	git branch feature-test
7	Pindah branch	git checkout feature-test
8	Buat test.txt + commit	echo ... > test.txt; git add test.txt; git commit -m "test: add test file"
9	Push feature-test	git push origin feature-test
10	Merge ke main	git checkout main; git merge feature-test; git push origin main

Blok perintah siap salin

```
# 1 via web GitHub
git clone https://github.com/diva-28-ukay/latihan-git-1224026.git
cd latihan-git-1224026
echo "# latihan-git-1224026" > README.md
echo "Nama: Diva Anastasya" >> README.md
echo "NIM: 1224026" >> README.md
git add README.md
git commit -m "docs: add README"
git push origin main
git branch feature-test
git checkout feature-test
echo "File test ..." > test.txt
git add test.txt
git commit -m "test: add test file"
git push origin feature-test
git checkout main
git merge feature-test
git push origin main
```

Bukti di repo ini

```
* e274db7 docs: add Tugas04 PDF
* 6346e08 feat: add unit tests, bank account, and java workflow
* 6260e8b test: add test file
* 0bce552 docs: add README
* bfce8da Initial commit
Branches: main, feature-test, origin/main, origin/feature-test
```

Verifikasi: git log --oneline --graph --all dan git branch -a

BAGIAN B: UNIT TESTING FRAMEWORK (35 POIN)

Soal 3 (15 Poin) - Validasi Pendaftaran ke JUnit

Kode acuan (PendaftaranValidator.java)

```
public String validasiPendaftaran(String nama, int umur, String email, boolean setujuSyarat) {
  if (nama == null || nama.isEmpty()) return "Nama harus diisi";        // P1
  if (umur < 17) return "Umur minimal 17 tahun";                        // P2
  if (email == null || !email.contains("@")) return "Email tidak valid"; // P3,P4
  if (!setujuSyarat) return "Harus menyetujui syarat dan ketentuan";    // P5
  return "Pendaftaran berhasil";
}
```

1. Lima test case (P1-P5 + sukses)

ID	Path	Input	Ekspektasi
TC1	P1 nama kosong	("",20,"budi@mail.com",true) + (null,...)	Nama harus diisi
TC2	P2 umur<17	("Budi",16,"budi@mail.com",true)	Umur minimal 17 tahun
TC3	P3/P4 email invalid	(null) dan ("budimail.com")	Email tidak valid
TC4	P5 tidak setuju	("Budi",20,"budi@mail.com",false)	Harus menyetujui...
TC5	Sukses	("Budi",20,"budi@mail.com",true)	Pendaftaran berhasil

Catatan: P3 (null) dan P4 (tanpa @) satu kondisi OR, digabung satu grup tapi diuji 2 assert agar cover 100%.

2. JUnit (PendaftaranValidatorTest.java, JUnit 5)

```
private final PendaftaranValidator v = new PendaftaranValidator();
@Test          void          testNamaKosong(){          assertEquals("Nama          harus          diisi",
v.validasiPendaftaran("",20,"budi@mail.com",true)); }
@Test          void          testNamaNull(){          assertEquals("Nama          harus          diisi",
v.validasiPendaftaran(null,20,"budi@mail.com",true)); }
@Test         void         testUmur(){         assertEquals("Umur         minimal         17         tahun",
v.validasiPendaftaran("Budi",16,"budi@mail.com",true)); }
@Test          void          testEmailNull(){          assertEquals("Email          tidak          valid",
v.validasiPendaftaran("Budi",20,null,true)); }
@Test         void         testEmailTanpaAt(){         assertEquals("Email         tidak         valid",
v.validasiPendaftaran("Budi",20,"budimail.com",true)); }
@Test    void    testTidakSetuju(){    assertEquals("Harus    menyetujui    syarat    dan    ketentuan",
v.validasiPendaftaran("Budi",20,"budi@mail.com",false)); }
@Test            void            testSukses(){            assertEquals("Pendaftaran            berhasil",
v.validasiPendaftaran("Budi",20,"budi@mail.com",true)); }
```

Run: mvn test -Dtest=PendaftaranValidatorTest (file lengkap 7 @Test ada di repo).

Soal 4 (20 Poin) - StudentGrade

1. Cyclomatic Complexity calculateGrade() = 7

Rumus V(G) = jumlah predicate + 1. Operator || dihitung 2 (short-circuit).

Kondisi	Predicate
if (score < 0 || score > 100)	2 (<0 dan >100)
if (score >= 80)	1

Kondisi	Predicate
else if (score >= 70)	1
else if (score >= 60)	1
else if (score >= 50)	1
TOTAL	6 -> V(G) = 6 + 1 = 7

2. Independent Path (7)

Path	Kondisi	Hasil
P1	score < 0 (mis -1)	throw IllegalArgumentException
P2	score > 100 (mis 101)	throw IllegalArgumentException
P3	score >= 80 (mis 85)	"A"
P4	70-79 (mis 75)	"B"
P5	60-69 (mis 65)	"C"
P6	50-59 (mis 55)	"D"
P7	0-49 (mis 40)	"E"

3. JUnit calculateGrade + isPassed (StudentGradeTest.java)

```
@Test void testGradeA(){ assertEquals("A", grade.calculateGrade(85)); assertEquals("A", grade.calculateGrade(80)); }
@Test void testGradeB(){ assertEquals("B", grade.calculateGrade(75)); }
@Test void testGradeC(){ assertEquals("C", grade.calculateGrade(65)); }
@Test void testGradeD(){ assertEquals("D", grade.calculateGrade(55)); }
@Test void testGradeE(){ assertEquals("E", grade.calculateGrade(40)); }
@Test void testInvalid(){ assertThrows(IllegalArgumentException.class, ()->grade.calculateGrade(-1));
  assertThrows(IllegalArgumentException.class, ()->grade.calculateGrade(101)); }
@Test void testPassedTrue(){ assertTrue(grade.isPassed(60)); assertTrue(grade.isPassed(80)); }
@Test void testPassedFalse(){ assertFalse(grade.isPassed(59)); assertFalse(grade.isPassed(0)); }
@Test void testPassedInvalid(){ assertThrows(IllegalArgumentException.class, ()->grade.isPassed(-5)); }
```

File repo lebih lengkap: boundary 100/79/70/60/50/0 + isPassed 100/59. Run: mvn test -Dtest=StudentGradeTest

BAGIAN C: GITHUB ACTIONS (25 POIN)

Soal 5 (15 Poin) - Analisis workflow Python CI

1. Nama workflow

Python CI (dari baris name: Python CI).

2. Trigger

```
on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
```

- push ke branch main dan develop
- pull_request ke branch main

3. Fungsi strategy.matrix

Menjalankan job yang SAMA secara PARALEL untuk beberapa kombinasi tanpa duplikasi YAML. Di sini python-version ['3.10','3.11','3.12'] berarti job test jalan 3x di ubuntu-latest, masing-masing satu versi. Akses via ${{ matrix.python-version }}. Tujuan: pastikan kompatibel lintas versi.

4. Steps di job test (koreksi: tertulis 5, faktanya 4)

#	Step	Aksi / Perintah
1	Checkout code	actions/checkout@v4 (download kode)
2	Setup Python	actions/setup-python@v5 + matrix version
3	Install dependencies	pip install --upgrade pip; pip install pytest pytest-cov
4	Run tests	pytest --cov=src --cov-report=xml

Tidak ada step ke-5 di YAML tersebut; runs-on/strategy bukan step.

Soal 6 (10 Poin) - Workflow Java Unit Test

File: .github/workflows/java-unit-test.yml. Nama "Java Unit Test", trigger push main/develop + PR main, runner ubuntu-latest, 5 steps sesuai ketentuan.

```
name: Java Unit Test
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
```

```
          distribution: 'temurin'
          cache: maven
      - name: Run tests
        run: mvn test
      - name: Generate coverage report
        run: mvn jacoco:report
      - name: Upload coverage report
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco/
```

Step	Tujuan
1 Checkout	Ambil kode ke runner
2 Setup JDK 17	Install Temurin 17 + cache maven
3 mvn test	Jalankan JUnit
4 mvn jacoco:report	Generate target/site/jacoco/
5 Upload artifact	Simpan jacoco-report

BAGIAN D: CODE COVERAGE & INTEGRASI (15 POIN)

Soal 7 (15 Poin) - Analisis coverage

```
---------- coverage: platform linux, python 3.11 ----------
Name                  Stmts   Miss  Cover
src/calculator.py        10      0   100%
src/login_system.py      15      2    87%
src/payment.py           20      8    60%
src/utils.py              5      1    80%
TOTAL                    50     11    78%
```

Jawaban

No	Pertanyaan	Jawaban
1	Total statement?	50 (kolom Stmts TOTAL)
2	Total miss?	11
3	Paling rendah?	src/payment.py, 60% (8/20 tak tereksekusi)
4	Tingkatkan payment.py?	Lihat htmlcov; tambah test gagal/boundary/mock; target miss 8->0-2
5	78% cukup?	Belum ideal (standar 80-90%). Baseline ok, wajib naik.

Detail poin 4: buka pytest --cov=src --cov-report=html, cari baris merah di payment.py, tambah kasus declined/saldo kurang/input invalid/exception, boundary 0/limit, branch if/else belum kena, tambah integration test mock gateway, rerun hingga >=80%.

Detail poin 5: 78% lumayan tapi 11 statement tak teruji dan modul kritis payment hanya 60% = risiko bug produksi. Coverage tinggi pun bukan jaminan (tidak ukur kualitas assert). Terima sementara jika payment diuji manual, jangka panjang naik ke >=85% fokus payment + login_system.

BAGIAN E: STUDI KASUS TERPADU - BONUS (10 POIN)

Soal 8 - BankAccount lengkap

1. Kode (BankAccount.java)

```
public class BankAccount {
  private double balance;
  public BankAccount(){ this.balance = 0.0; }
  public BankAccount(double initial){
    if (initial < 0) throw new IllegalArgumentException("Initial balance cannot be negative");
    this.balance = initial;
  }
  public void deposit(double a){
    if (a <= 0) throw new IllegalArgumentException("Deposit amount must be positive");
    balance += a;
  }
  public void withdraw(double a){
    if (a <= 0) throw new IllegalArgumentException("Withdraw amount must be positive");
    if (a > balance) throw new IllegalArgumentException("Insufficient balance");
    balance -= a;
  }
  public double getBalance(){ return balance; }
}
```

2. Unit test (9 case, syarat min 6) - BankAccountTest.java

#	Test	Ekspektasi
1	testInitialBalanceZero	new BankAccount() -> 0.0
2	testInitialBalanceWithValue	new BankAccount(100) -> 100
3	testInitialNegativeThrows	new BankAccount(-50) -> throw
4	testDeposit	100 + deposit 50 -> 150
5	testDepositNegatifThrows	deposit -10 / 0 -> throw
6	testWithdraw	100 - withdraw 40 -> 60
7	testWithdrawOverThrows	withdraw 150 dari 100 -> throw
8	testWithdrawNegatifThrows	withdraw -20 -> throw
9	testTransaksi	deposit 200, withdraw 75 -> 125

Run: mvn test -Dtest=BankAccountTest atau mvn test (semua).

3. GitHub Actions

Pakai file yang sama dengan Soal 6: mvn test + mvn jacoco:report + upload jacoco-report. Memenuhi ketentuan Soal 8.

4. Repo & pipeline

- Link: https://github.com/diva-28-ukay/latihan-git-1224026
- Push: git push origin main + git push origin feature-test (sudah sukses 5 commit).
- Cek tab Actions > Java Unit Test harus hijau.
- Download artifact jacoco-report, buka index.html untuk screenshot coverage.
- Lampirkan: (1) screenshot Actions sukses, (2) screenshot coverage report.

LAMPIRAN: CARA JALAN + LINK

Menjalankan lokal (butuh JDK 17 + Maven)

```
mvn test
mvn jacoco:report
# laporan: target/site/jacoco/index.html
```

Link repository

https://github.com/diva-28-ukay/latihan-git-1224026

Checklist pengumpulan

- PDF ini: Tugas04_GitHubActions_1224026_DivaAnastasya.pdf (root + Tugas/Tugas04/)
- JAWABAN_TUGAS04.md (versi markdown)
- Kode + test + workflow sudah di-push (main & feature-test)
- Submit link GitHub ke LMS / Google Classroom

Disusun untuk Tugas Pertemuan 4, Bagian A s.d. E. Nama Diva Anastasya, NIM 1224026.